

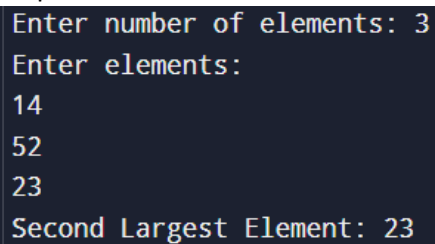
1.Second Largest Element

- Find the second largest element in a List<Integer>.

Code:

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();
        List < Integer > list = new ArrayList < > ();
        System.out.println("Enter elements:");
        for (int i = 0; i < n; i++) {
            list.add(sc.nextInt());
        }
        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;
        for (int num: list) {
            if (num > largest) {
                secondLargest = largest;
                largest = num;
            }
            else if (num > secondLargest && num != largest) {
                secondLargest = num;
            }
        }
        if (secondLargest == Integer.MIN_VALUE) {
            System.out.println("No second largest element");
        } else {
            System.out.println("Second Largest Element: " +
secondLargest);
        }
        sc.close();
    }
}
```

Output:

A screenshot of a terminal window showing the execution of the Java program. The user enters '3' for the number of elements, then enters the elements '14', '52', and '23' on separate lines. The program outputs 'Second Largest Element: 23'.

```
Enter number of elements: 3
Enter elements:
14
52
23
Second Largest Element: 23
```

2. Common Elements Between Two Sets

- Find intersection of two HashSet<Integer>

Code:

```
import java.util.*;

public class IntersectionOfSets {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        HashSet < Integer > set1 = new HashSet < > ();
        HashSet < Integer > set2 = new HashSet < > ();

        System.out.print("Enter number of elements in Set 1: ");
        int n1 = sc.nextInt();

        System.out.println("Enter elements of Set 1:");
        for (int i = 0; i < n1; i++) {
            set1.add(sc.nextInt());
        }

        System.out.print("Enter number of elements in Set 2: ");
        int n2 = sc.nextInt();

        System.out.println("Enter elements of Set 2:");
        for (int i = 0; i < n2; i++) {
            set2.add(sc.nextInt());
        }

        set1.retainAll(set2);
```

```

        System.out.println("Common Elements: " + set1);

        sc.close();
    }
}

```

Output:

```

Enter number of elements in Set 1: 3
Enter elements of Set 1:
10
11
12
Enter number of elements in Set 2: 4
Enter elements of Set 2:
8
9
10
11
Common Elements: [10, 11]

```

3. Symmetric Difference

- Find elements present in either of the sets but not both.

Code:

```

import java.util.*;

public class SymmetricDifference {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        HashSet < Integer > set1 = new HashSet < > ();
        HashSet < Integer > set2 = new HashSet < > ();

        System.out.print("Enter number of elements in Set 1: ");
        int n1 = sc.nextInt();

        System.out.println("Enter elements of Set 1:");
    }
}

```

```

for (int i = 0; i < n1; i++) {
    set1.add(sc.nextInt());
}

System.out.print("Enter number of elements in Set 2: ");
int n2 = sc.nextInt();

System.out.println("Enter elements of Set 2:");
for (int i = 0; i < n2; i++) {
    set2.add(sc.nextInt());
}

HashSet < Integer > union = new HashSet < > (set1);
union.addAll(set2);

HashSet < Integer > intersection = new HashSet < > (set1);
intersection.retainAll(set2);

union.removeAll(intersection);

System.out.println("Symmetric Difference: " + union);

sc.close();
}
}

```

Output:

```

Enter number of elements in Set 1: 3
Enter elements of Set 1:
10
11
12
Enter number of elements in Set 2: 4
Enter elements of Set 2:
7
9
10
11
Symmetric Difference: [7, 9, 12]

```